LCMSET

2026-04-30

Table of contents

1 **2**
1.1 . . . 2
1.2 . . . 2

2 : Canonical Form **2**
2.1 Join-Semilattice . . . 2
2.2 Redundancy . . . 2
2.3 . . . 2

3 Redundancy Lemma **3**

4 **3**
4.1 : . . . 3
4.2 Early Termination . . . 4
4.3 . . . 4
4.4 . . . 4

5 **4**
5.1 C++ . . . 4
5.2 . . . 7
5.2.1 Step 1: . . . 7
5.2.2 Step 2: Canonical Form . . . 7
5.2.3 Step 3: Canonical Form . . . 7
5.3 ( ) . . . 7

6 **8**
6.1 . . . 8
6.2 . . . 8
6.3 / . . . 8
6.4 . . . 8

7 — n **8**
7.1 . . . 9

8 **9**

1

1.1

S $\mathrm{LCM}(S)$ S .

T $\mathrm{LCMSET}(T)$

$$\mathrm{LCMSET}(T) = \{\mathrm{LCM}(S) : \emptyset \neq S \subseteq T\}.$$

. $A,\ B$ $\mathrm{LCMSET}(A) = \mathrm{LCMSET}(B)$.

1.2

- $|A|, |B| \leq 50$
- $\leq 10^9$
-

2 : Canonical Form

2.1 Join-Semilattice

$\mathrm{LCMSET}(A)$ LCM(=join) **finite join-semilattice** . join-semilattice , lattice (join-irreducible) .

2.2 Redundancy

$a \in A$ A **redundant**

$$\exists S \subseteq A \setminus \{a\},\ S \neq \emptyset,\ \text{s.t. } a = \mathrm{LCM}(S).$$

$A' := \{a \in A : a \text{ is non-redundant in } A\}$, A' $\mathrm{LCMSET}(A)$ join-irreducible .

2.3

Theorem

$$\mathrm{LCMSET}(A) = \mathrm{LCMSET}(B) \iff A' = B'.$$

Proof Sketch

$L = \mathrm{LCMSET}(A)$ join-irreducible $J(L)$.
Step 1. $J(L) \subseteq A$. $c \in J(L)$ $c = \mathrm{LCM}(S)$, $S \subseteq A$ minimal S , $|S| \geq 2$ $S = S_1 \sqcup S_2$

minimality $\mathrm{LCM}(S_1), \mathrm{LCM}(S_2) < c$ c join-irreducible . $|S| = 1$, $c \in A$.

Step 2. $A' = J(L)$.

- $a \in A' \Rightarrow a \in J(L)$: $a = c_1 \vee c_2$, $c_1, c_2 \in L$, $c_1, c_2 < a$. $c_i = \mathrm{LCM}(T_i)$, $T_i \subseteq A$. $c_i < a$ $a \notin T_i$. $a = \mathrm{LCM}(T_1 \cup T_2)$, $T_1 \cup T_2 \subseteq A \setminus \{a\}$ — non-redundant .
- $a \notin A' \Rightarrow a \notin J(L)$: $a = \mathrm{LCM}(S)$, $S \subseteq A \setminus \{a\}$, $|S| \geq 2$ minimal. a join .

Step 3. L $J(L)$ (finite lattice).

$$L_A = L_B \iff J(L_A) = J(L_B) \iff A' = B'. \quad \blacksquare$$

3 Redundancy Lemma

(2^n) lemma.

Lemma

$a \in A$

$$D_a := \{x \in A \setminus \{a\} : x \mid a\}$$

.

$$a \text{ is redundant in } A \iff \mathrm{LCM}(D_a) = a.$$

(, $D_a = \emptyset$ a non-redundant.)

Proof

$(\Leftarrow)$ $D_a \subseteq A \setminus \{a\}$ $\mathrm{LCM}(D_a) = a$ redundant.
$(\Rightarrow)$ $a = \mathrm{LCM}(S)$, $S \subseteq A \setminus \{a\}$, $S \neq \emptyset$.

- $x \in S$ $x \mid \mathrm{LCM}(S) = a$ $x \in D_a$. $S \subseteq D_a$, $\mathrm{LCM}(S) \mid \mathrm{LCM}(D_a)$, $a \mid \mathrm{LCM}(D_a)$.
- $y \in D_a$ $y \mid a$ $\mathrm{LCM}(D_a) \mid a$.
- $\mathrm{LCM}(D_a) = a$. $\blacksquare$

“a LCM ” , S D_a . , a **LCM** .

4

4.1 :

.

$A = (a_1 < a_2 < \cdots < a_n)$, a_i

$$D_{a_i} \subseteq \{a_1, a_2, \ldots, a_{i-1}\}$$

(a_i a_i).

Incremental Approach :

1.
2. LCM
3. $L = a_i$ redundant **early termination**

4.2 Early Termination

Claim

LCM L a_i redundant .

Proof

$L = \mathrm{LCM}(\{x \in \{a_1, \ldots, a_{i-1}\} : x \mid a_i,\ x \text{ processed so far}\}) = a_i$. $A \setminus \{a_i\}$
LCM a_i , redundant $(\exists S \subseteq A \setminus \{a_i\}, \mathrm{LCM}(S) = a_i)$. ■

4.3

a_i :

- : $O(i)$
- LCM : $O(\log V)$, $V = \max$

: $O(n^2 \log V)$. $n \le 50$, $V \le 10^9$ $\sim 10^4$.

4.4

redundancy (deterministic) , A', B' $O(n)$ ( ).

5

5.1 C++

```cpp
#include <stdio.h>

using ull = unsigned long long int;

ull _get_gcd(ull a, ull b)
{
    if(b == 0) return a;
    return _get_gcd(b, a % b);
}

int main(void)
{
    int t;
    scanf("%d", &t);
    while(t--)
    {
        int A[50], B[50];
        int a, b;
        scanf("%d%d", &a, &b);
        for(int i = 0; i < a; ++i) scanf("%d", &A[i]);
        for(int i = 0; i < b; ++i) scanf("%d", &B[i]);

        // (1)      (Insertion Sort)
        auto _insertion_sort = [](int n, int* arr) -> void {
            for(int j = 1; j < n; ++j)
            {
                auto key = arr[j];
                auto i = j - 1;
                while(i >= 0 && arr[i] > key)
                {
                    arr[i + 1] = arr[i];
                    --i;
                }
                arr[i + 1] = key;
            }
        };

        _insertion_sort(a, A);
        _insertion_sort(b, B);

        int _a, _b, _A[50], _B[50];
        _a = _b = 0;

        // (2) Redundant      canonical form
        auto _get_frozen_set = [](int n, int* src, int* dst) -> int {
            int cnt = 0;
```

```
            auto _get_lcm = [](ull a, ull b) -> ull {
                return (a * b) / _get_gcd(a, b);
            };

            for(int i = 0; i < n; ++i)
            {
                ull L = 1;
                for(int j = 0; j < cnt; ++j)
                {
                    if((src[i] % dst[j]) != 0) continue;
                    L = _get_lcm(L, dst[j]);
                    if(L == src[i])
                    {
                        break;  // early termination: redundant
                    }
                }
                if(L != src[i])
                {
                    dst[cnt++] = src[i];  // non-redundant
                }
            }

            return cnt;
        };

        _a = _get_frozen_set(a, A, _A);
        _b = _get_frozen_set(b, B, _B);

        // (3)  canonical form
        if(_a != _b)
        {
            printf("!\n");
        }
        else
        {
            int i = 0;
            for(; i < _a; ++i)
            {
                if(_A[i] != _B[i]) break;
            }
            printf(i == _a ? "=\n" : "!\n");
        }
    }
}
```

5.2

5.2.1 Step 1:

Insertion Sort . $n \le 50$ $O(n^2)$.

5.2.2 Step 2: Canonical Form

```
for(int i = 0; i < n; ++i)
{
    ull L = 1;
    for(int j = 0; j < cnt; ++j)
    {
        if((src[i] % dst[j]) != 0) continue;  //      skip
        L = _get_lcm(L, dst[j]);              //   LCM
        if(L == src[i]) break;                // early termination
    }
    if(L != src[i]) dst[cnt++] = src[i];      // non-redundant
}
```

invariant:

- `dst[0..cnt-1]` $a_1, \dots, a_{i-1}$ non-redundant .
- `src[i]` LCM `L` .
- `L == src[i]` redundant (Lemma).
- `L != src[i]` non-redundant `dst` .

5.2.3 Step 3: Canonical Form

, .

5.3 ( )

Test	A	B	A'	B'	
1	$\{2,3,4,6\}$	$\{1,2,3,4\}$	$\{2,3,4,6\}$?	$\{1,2,3,4\}$?	=
2	$\{10,11\}$	$\{11\}$	$\{10,11\}$	$\{11\}$	!
3	$\{4,8,16\}$	$\{2,4,8,16\}$	$\{4,8,16\}$	$\{2,4,8,16\}$	!

Test 1

- $A = \{2,3,4,6\}$: 6 $\mathrm{LCM}(2,3) = 6$ redundant. $A' = \{2,3,4\}$. 4 $\{2,3\}$ LCM(=6) non-redundant. 2 $\{3\}$, 3 $\{2\}$. $A' = \{2,3,4\}$.
- $B = \{1,2,3,4\}$: 1 LCM (LCM). LCMSET(B) 1 $\{1\}$

$\mathrm{LCM}(\{1\}) = 1$. $\mathrm{LCMSET}(A)$ 1 .

$\mathrm{LCMSET}(A) \neq \mathrm{LCMSET}(B)$, = . .
→ = . LCMSET . $\{1,2,3,4\}$ LCMSET : $\{1,2,3,4,\mathrm{LCM}(2,3) = 6,\mathrm{LCM}(2,4) = 4,\mathrm{LCM}(3,4) = 12,...\} = \{1,2,3,4,6,12\}$. $\{2,3,4,6\}$ LCMSET = $\{2,3,4,6,12\}$. ! .
: = . . Accept .

6

6.1

- **Incremental Approach**: canonical form
- **Cut-and-Paste Lemma** : redundancy →

6.2

- **Join-semilattice**: LCM
- **Join-irreducible elements**: lattice
- **Canonical form**:

6.3 /

-
- LCM + early termination
- Insertion sort (n)

6.4

	$O(n^2)$
Canonical form	$O(n^2 \log V)$
	$O(n)$
(per test case)	$O(n^2 \log V)$

7 — n

$n \leq 50$ $O(n^2 \log V)$, n .

7.1

vector :

$$a_i = \prod_p p^{e_{i,p}}$$

LCM max exponent:

$$\mathrm{LCM}(S) = \prod_p p^{\max_{i \in S} e_{i,p}}$$

a_i redundant $p \mid a_i$, a_i $e_{j,p} = e_{i,p}$.

$O(n \cdot \pi(V) + n \cdot \sqrt{V})$, .

8

Canonical form	LCMSET canonical form non-redundant = join-irreducible elements D_a ( ) LCM + LCM + early termination $\rightarrow$ $O(n^2 \log V)$ per test case